

QUESTION 1

QUESTION 1

Reverse the NumPy array: `arr = np.array([1, 2, 3, 6, 4, 5])`

SOLUTION

```
arr = np.array([1, 2, 3, 6, 4, 5])
arr = arr[::-1]
print(arr)
```

OUTPUT

```
[5 4 6 3 2 1]
```

QUESTION 2

Flatten the NumPy arr: `array1 = np.array([[1, 2, 3], [2, 4, 5], [1, 2, 3]])` using any two NumPy in-built methods

SOLUTION

```
arr = np.array([[1,2,3], [2, 4, 5], [1, 2, 3]])
arr = arr.flatten()
print(arr)
```

OUTPUT

```
[1 2 3 2 4 5 1 2 3]
```

QUESTION 3

Compare the following numpy arrays

```
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[1, 2], [3, 4]])
```

SOLUTION

```
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[1, 2], [3, 4]])
```

OUTPUT

```
[[ True  True]
 [ True  True]]
```

```
print(arr1 == arr2)
```

QUESTION 4

Find the most frequent value and their indice(s) in the following arrays:

- i. `x = np.array([1,2,3,4,5,1,2,1,1,1])`
- ii. `y = np.array([1, 1, 1, 2, 3, 4, 2, 4, 3, 3, ])`

SOLUTION

```
x = np.array([1, 2, 3, 4, 5, 1, 2, 1, 1, 1])
y = np.array([1, 2, 1, 2, 3, 4, 2, 4, 3, 3])
```

OUTPUT

```
1
3
```

```
counts = {}
for i in x:
    counts[i] = counts.get(i, 0) + 1
print(max(counts, key=counts.get))
```

```
counts = {}
for i in y:
```

```
counts[i] = counts.get(i, 0) + 1
print(max(counts, key=counts.get))
```

QUESTION 5

For the array `gfg = np.matrix('[4, 1, 9; 12, 3, 1; 4, 5, 6]')`, find

- Sum of all elements
- Sum of all elements, row-wise
- Sum of all elements, column-wise

SOLUTION

```
gfg = np.matrix('[4, 1, 9; 12, 3, 1; 4, 5, 6]')
print(np.sum(gfg))
print(np.sum(gfg, axis=0))
print(np.sum(gfg, axis=1))
```

OUTPUT

```
45
[[20  9 16]]
[[14]
 [16]
 [15]]
```

QUESTION 6

For the matrix: `n_array = np.array([[55, 25, 15],[30, 44, 2],[11, 45, 77]])`, find

- Sum of diagonal elements
- Eigen values of matrix
- Eigen vectors of matrix
- Inverse of matrix
- Determinant of matrix

SOLUTION

```
n_array = np.array([[55, 25, 15], [30, 44, 2], [11, 45, 77]])
print(np.trace(n_array))
eigenvalues, eigenvectors = np.linalg.eig(n_array)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
inverse = np.linalg.inv(n_array)
print("Inverse of the matrix:\n", inverse)
det = np.linalg.det(n_array)
```

OUTPUT

QUESTION 7

Multiply the following matrices and also find covariance between matrices using NumPy:

- `p = [[1, 2], [2, 3]]`
`q = [[4, 5], [6, 7]]`
- `p = [[1, 2], [2, 3], [4, 5]]`
`q = [[4, 5, 1], [6, 7, 2]]`

SOLUTION

```
p = np.array([[1, 2], [2, 3]])
q = np.array([[4, 5], [6, 7]])

product = np.multiply(p, q)
covariance = np.cov(p, q, rowvar=False)
```

OUTPUT

```
print( product)
print(covariance)
```

QUESTION 8

For the matrices: $x = \text{np.array}([[2, 3, 4], [3, 2, 9]])$; $y = \text{np.array}([[1, 5, 0], [5, 10, 3]])$, find inner, outer and cartesian product?

SOLUTION

```
x = np.array([[2, 3, 4], [3, 2, 9]])
y = np.array([[1, 5, 0], [5, 10, 3]])

inner_product = np.inner(x, y)
outer_product = np.outer(x, y)

print("Inner Product:\n", inner_product)
print("Outer Product:\n", outer_product)
```

OUTPUT

QUESTION 2

QUESTION 9

For the array: `array = np.array([[1, -2, 3], [-4, 5, -6]])`

- Find element-wise absolute value
- Find the 25th, 50th, and 75th percentile of flattened array, for each column, for each row.
- Mean, Median and Standard Deviation of flattened array, of each column, and each row

SOLUTION

```
arr = np.array([[1, -2, 3], [-4, 5, -6]])

abs_values = np.abs(arr)
print(abs_values)

perc_25_row = np.percentile(arr, 25, axis=1)
perc_25_col = np.percentile(arr, 25, axis=0)
print(perc_25_row)
print(perc_25_col)

perc_50_row = np.percentile(arr, 50, axis=1)
perc_50_col = np.percentile(arr, 50, axis=0)
print(perc_50_row)
print(perc_50_col)

perc_75_row = np.percentile(arr, 75, axis=1)
perc_75_col = np.percentile(arr, 75, axis=0)
print(perc_75_row)
print(perc_75_col)

flattened_arr = arr.flatten()
print(np.mean(flattened_arr))
print(np.median(flattened_arr))
print(np.std(flattened_arr))
```

OUTPUT

QUESTION 10

For the array: `a = np.array([-1.8, -1.6, -0.5, 0.5, 1.6, 1.8, 3.0])`. Find floor, ceiling and truncated value, rounded values

SOLUTION

```
a = np.array([-1.8, -1.6, -0.5, 0.5, 1.6, 1.8, 3.0])
floor_values = np.floor(a)
ceil_values = np.ceil(a)
trunc_values = np.trunc(a)
rounded_values = np.round(a)

print(floor_values)
print(ceil_values)
print(trunc_values)
print(rounded_values)
```

OUTPUT

QUESTION 3

QUESTION 11

For the array: `array = np.array([10, 52, 62, 16, 16, 54, 453])`, find

- i. Sorted array
- ii. Indices of sorted array
- iii. 4 smallest elements
- iv. 5 largest elements

SOLUTION

```
arr = np.array([10, 52, 62, 16, 16, 54, 453])
sorted_arr = np.sort(arr)
```

```
print(sorted_arr)
print(sorted_arr[:4])
print(sorted_arr[::-1][:5])
```

OUTPUT

QUESTION 12

For the array: `array = np.array([1.0, 1.2, 2.2, 2.0, 3.0, 2.0])`, find

- i. Integer elements only
- ii. Float elements only

SOLUTION

```
arr = np.array([1.0, 1.2, 2.2, 2.0, 3.0, 2.0])
print(arr)
```

```
arr_int = [i for i in arr if i.is_integer()]
print(arr_int)
```

```
arr_float = [i for i in arr if i.is_integer() is False]
print(arr_float)
```

OUTPUT

QUESTION 4

QUESTION 13

Write a function named `img_to_array(path)` that reads an image from a specified path and save it as text file on local machine? (Note: use separate cases for RGB and Grey Scale images)

SOLUTION

```
import numpy as np
import cv2

def img_to_array(path):
    img = cv2.imread(path)
    with open(f"{path}.txt", "a+") as f:
        f.write(str(img))

img_to_array("./screenshots.png")
```

OUTPUT